Unit 2
Unit 2: JavaScript, AJAX & jQuery — Practical/Oral Exam Notes

Context

Part of Web Application Development | Practical + Oral Exam Prep | SPPU Third Year IT

High-Yield Oral Topics

- **var vs let vs const** — Always asked in every viva
- **Promises vs Async/Await** — Advanced JavaScript must-know
- **AJAX flow** — How data is sent/received without page reload
- **JSON** — Data format between frontend and backend
- **Arrow Functions** — Syntax difference + `this` behavior

MINI PROJECT ANCHOR: Student Management System (SMS)

> All JavaScript examples manage student data: register, list, validate, send via AJAX.

SECTION 1 — JAVASCRIPT FUNDAMENTALS

Exam Definition

JavaScript is a **client-side scripting language** (also used server-side with Node.js) that adds **interactivity and dynamic behavior** to web pages. It runs directly in the browser.

var vs let vs const — MOST ASKED VIVA QUESTION

Feature	`var`	`let`	`const`
Scope	Function scope	Block scope `{}`	Block scope `{}`
Re-declare	✅ Yes	X No	X No
Re-assign	✅ Yes	✅ Yes	X No
Hoisting	Hoisted (undefined)	Hoisted (TDZ error)	Hoisted (TDZ error)
Best For	Avoid using	Variables that change	Constants

> **TDZ = Temporal Dead Zone** — `let` / `const` can't be accessed before declaration.

Example concepts:

- `var`: Function-scoped, can be re-declared and re-assigned.
- `let`: Block-scoped, cannot be re-declared, can be re-assigned.
- `const`: Block-scoped, cannot be re-declared or re-assigned (though object properties can change).

Data Types in JavaScript

Type	Example	typeof
Number	`42`, `3.14`	`"number"`
String	`"Hello"`	`"string"`
Boolean	`true`, `false`	`"boolean"`
Undefined	`let x;`	`"undefined"`
Null	`let x = null`	`"object"` (JS bug!)
Object	`{ name: "Rahul" }`	`"object"`
Array	`[1, 2, 3]`	`"object"`
Function	`function foo(){}`	`"function"`

> **Viva Trap:** `typeof null` returns `"object"` — this is a known JavaScript bug. Correct answer: null is a primitive, not an object.

Functions — Three Ways

- **Function Declaration**: Hoisted, can be called before definition.
- **Function Expression**: Stored in a variable, not hoisted.
- **Arrow Function**: Short syntax, inherits `this` from parent scope.

Arrays — Key Methods for SMS

Key methods to know:

- `forEach()`: Iterates over elements.
- `map()`: Creates a new array by transforming elements.
- `filter()`: Creates a new array with elements passing a test.
- `find()`: Returns the first element that passes a test.
- `push()`: Adds element to end.
- `splice()`: Removes or replaces elements.

Objects — Key for SMS

Key concepts:

- **Creation**: Using braces `{ key: value }`.
- **Access**: Dot notation (`obj.prop`) or bracket notation (`obj["prop"]`).

- **Methods**: Functions stored as properties.
- **Destructuring**: Extracting values into variables easily.
- **Spread Operator**: `...` used to copy or merge objects.

Variable Scope — Viva Explained

```
let globalVar = "I'm global";    // Accessible everywhere

function testScope() {
  let localVar = "I'm local";   // Only inside function
  console.log(globalVar);       // ✅ Works
  console.log(localVar);        // ✅ Works
}

console.log(localVar);          // ❌ ReferenceError
```

🎤 VIVA Q&A — JavaScript Basics

Q1: What is the difference between `==` and `===`?

> `==` compares values only (type coercion happens): `"5" == 5` is `true`.
> `===` compares both value AND type (strict): `"5" === 5` is `false`.
> Always use `===` in professional code.

Q2: What is hoisting?

> Hoisting is JavaScript's behavior of moving **declarations** to the top of the scope before code executes. `var` declarations are hoisted with value `undefined`. Function declarations are fully hoisted (callable before they appear). `let` and `const` are hoisted but in the Temporal Dead Zone (throw error if accessed early).

Q3: What is the difference between `null` and `undefined`?

> `undefined`: Variable declared but no value assigned (by JavaScript). `null`: Intentionally set to "no value" by the developer. `typeof undefined` = `"undefined"`, `typeof null` = `"object"` (JS bug).

SECTION 2 — ADVANCED JAVASCRIPT

JSON — JavaScript Object Notation

📌 Exam Definition

JSON is a **lightweight data interchange format** based on JavaScript object syntax. It is used to transmit data between frontend and backend as plain text. It is **language-independent** and human-readable.

Key operations:

- `JSON.stringify()`: Converts a JavaScript object into a JSON string.
- `JSON.parse()`: Converts a JSON string back into a JavaScript object.

> **Key Rule:** JSON keys MUST be in **double quotes**. No functions, no `undefined`. Only: strings, numbers, booleans, arrays, objects, null.

Arrow Functions — Full Explanation

Syntax rules:

- Omit `()` if there is only one parameter.
- Omit `{}` and `return` for single-line implicit returns.
- Always use `()` if there are no parameters.

> **Critical Difference:** Arrow functions do NOT have their own `this`. They inherit `this` from the parent scope. Regular functions have their own `this`.

Callback Functions

A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.

Promises — Solving Callback Hell

A Promise is an object representing the eventual completion or failure of an asynchronous operation. Use `.then()` for success and `.catch()` for error handling.

Async/Await — Modern Way

The `async` and `await` keywords enable asynchronous, promise-based behavior to be written in a cleaner style, avoiding the need to explicitly configure promise chains.

Promises vs Async/Await — Comparison Table

Feature	Promises	Async/Await
Syntax	`.then().catch()` chaining	`async/await` with try/catch
Readability	Can get messy with chains	Clean, looks synchronous
Error Handling	`.catch()`	`try/catch` block
Debugging	Hard to trace in stack	Easier to debug
Under the Hood	Async/Await IS Promises	Built on top of Promises

> **One-liner answer:** "Async/Await is syntactic sugar over Promises. It makes asynchronous code look like synchronous code, improving readability."

Error Handling

Use `try...catch` blocks to handle runtime errors in JavaScript. The `finally` block executes regardless of the try/catch outcome.

🎤 VIVA Q&A — Advanced JavaScript

Q1: What is a Promise in JavaScript?

> A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: **Pending** (waiting), **Fulfilled** (success — resolved), and **Rejected** (failed). Use `.then()` for success, `.catch()` for errors.

Q2: What is the difference between Callback and Promise?

> Callbacks pass a function to be called later, but lead to "Callback Hell" when nested. Promises chain `.then()` calls in a flat, readable way. Both handle async operations, but Promises are cleaner and have better error handling via `.catch()`.

Q3: Why use Async/Await?

> Async/Await makes asynchronous code look and behave like synchronous code. It eliminates promise chaining, making code cleaner and easier to debug. Errors are handled with standard try/catch instead of `.catch()`.

Q4: What is JSON and why is it used?

> JSON (JavaScript Object Notation) is a lightweight, text-based data format for transmitting data between client and server. It's language-independent, easy to read, and natively supported in JavaScript via `JSON.stringify()` and `JSON.parse()`.

Q5: What is an arrow function and how is it different?

> Arrow functions are a shorter syntax for writing functions. Key difference: they do NOT have their own `this` keyword — they inherit it from the enclosing scope. This solves common `this` binding bugs in callbacks.

⚠️ Oral Trap Questions

Trap: "Can `const` objects be modified?"

> Yes! `const` prevents reassignment of the variable, but the object's properties can still be modified. `const student = {}; student.name = "Rahul";` — This is valid. To fully freeze an object, use `Object.freeze(student)`.

Trap: "Is JavaScript single-threaded?"

> Yes! JavaScript runs on a single thread. But it handles asynchronous operations through the **Event Loop**, **Callback Queue**, and **Web APIs** (browser provides these). This is why async operations don't block the UI.

SECTION 3 — AJAX

📌 Exam Definition

AJAX (Asynchronous JavaScript And XML) is a technique that allows web pages to **send and receive data from a server in the background** without reloading the entire page. Despite the name, it commonly uses **JSON** today, not XML.

Why AJAX?

- Without AJAX: Click button → Full page reload → Bad UX
- With AJAX: Click button → Only data updates → Smooth UX

AJAX Full Flow (Frontend → Backend → Response)

```
USER ACTION (Click Submit)
        ↓
JavaScript creates XMLHttpRequest / fetch()
        ↓
HTTP Request sent to server (GET/POST/PUT/DELETE)
        ↓
Server processes request (Node.js/Express)
        ↓
Database query (MongoDB)
        ↓
Server sends JSON response
        ↓
JavaScript receives response (callback / .then())
        ↓
DOM updated without page reload ✅
```

AJAX with fetch() — Modern Way (Most Used in Practical)

The `fetch()` API is the modern way to make AJAX requests. It returns a Promise.

- **GET:** `fetch(url)`
- **POST:** `fetch(url, { method: 'POST', headers: {...}, body: JSON.stringify(data) })`

AJAX with XMLHttpRequest — Old Way (May Ask in Viva)

`XMLHttpRequest` is the traditional way to perform AJAX before `fetch` was introduced. It uses callbacks and tracks state changes (0-4).

HTTP Status Codes — Viva Must Know

Code	Meaning	When Used
`200`	OK	Successful GET/PUT
`201`	Created	Successful POST (new resource created)
`400`	Bad Request	Invalid data sent
`401`	Unauthorized	Not logged in
`403`	Forbidden	Logged in but no permission
`404`	Not Found	Resource doesn't exist
`500`	Internal Server Error	Server-side bug

🎤 VIVA Q&A — AJAX

Q1: What is AJAX and why is it used?

> AJAX (Asynchronous JavaScript and XML) is a technique to communicate with a server without reloading the page. It allows web apps to update parts of the UI dynamically by sending/receiving data in the background. Example: When you search on Google, suggestions appear without page reload.

Q2: What HTTP methods does AJAX support?

> AJAX supports all HTTP methods: **GET** (retrieve data), **POST** (send/create data), **PUT** (update data), **DELETE** (remove data). In the `fetch()` API, you specify the method in the options object.

Q3: What is the difference between synchronous and asynchronous AJAX?

> Synchronous AJAX blocks the browser (page freezes) until response arrives. Asynchronous AJAX (recommended) sends request and continues executing; a callback runs when data arrives. Always use asynchronous to keep the UI responsive.

Q4: What is CORS?

> CORS (Cross-Origin Resource Sharing) is a security feature. Browsers block AJAX requests to a different domain by default. To allow it, the server must send `Access-Control-Allow-Origin` headers. Example: Frontend on port 3000, Backend on port 5000 — CORS must be enabled on backend.

SECTION 4 — JQUERY

📌 Exam Definition

jQuery is a **fast, lightweight JavaScript library** that simplifies **DOM manipulation, event handling, CSS animation, and AJAX calls** with a shorter, cross-browser compatible syntax.

Why jQuery?

Task	Vanilla JavaScript	jQuery
Select element	`document.getElementById('btn')`	`$('#btn')`
Add event listener	`element.addEventListener('click', fn)`	`$('#btn').click(fn)`
Change text	`element.textContent = 'Hello'`	`$('#el').text('Hello')`
Hide element	`element.style.display = 'none'`	`$('#el').hide()`
AJAX GET	`fetch()` with complex options	`$.get(url, callback)`

SMS Practical with jQuery

Core jQuery operations for the exam:

- **Selectors**: `$('#id')`, `$('.class')`
- **Events**: `$(element).click(fn)`, `$(element).submit(fn)`
- **DOM**: `$(element).append(content)`, `$(element).remove()`
- **AJAX**: `$.ajax({ url, method, success: fn })`
- **Effects**: `$(element).slideToggle()`, `$(element).fadeOut()`

🎤 VIVA Q&A — jQuery

Q1: What is jQuery? Why do we use it?
> jQuery is a JavaScript library that simplifies DOM manipulation, event handling, and AJAX. The `$` sign is its alias. We use it because it reduces code length, handles cross-browser differences automatically, and makes complex operations like animations and AJAX very simple.

Q2: What is `$(document).ready()`?
> It ensures the code inside runs only AFTER the entire HTML DOM is fully loaded. Without it, your script might try to access elements that don't exist yet. It's equivalent to `window.onload` but more efficient (doesn't wait for images).

Q3: What is DOM manipulation?
> DOM (Document Object Model) is the tree structure that browsers create from HTML. DOM manipulation is changing this structure dynamically using JavaScript/jQuery. Examples: Adding new table rows, changing text content, showing/hiding elements, adding/removing CSS classes.

Q4: What is the difference between `$(this)` and `this` in jQuery?
> `this` refers to the native DOM element. `$(this)` wraps it in a jQuery object, making all jQuery methods available. You need `$(this)` to use methods like `.text()`, `.hide()`, `.addClass()` etc.

jQuery vs Vanilla JS

Feature	Vanilla JS	jQuery
Performance	Faster (no library overhead)	Slightly slower
Code Length	Verbose	Concise
Browser Support	Must handle manually	Handled automatically
Dependencies	None	Requires jQuery file
Modern Apps	Preferred (with React/Angular)	Legacy/simple projects

> **When asked "Why not jQuery in modern apps?":** Modern frameworks like React and Angular handle DOM updates more efficiently through Virtual DOM and change detection. jQuery is fine for simple projects and legacy codebases.

📋 PRACTICAL FILE EXPECTATIONS — Assignment 1 (JavaScript Part)

Your JavaScript code should demonstrate:
1. Form validation with clear error messages
2. Storing student data in array + localStorage
3. Displaying students in HTML table dynamically
4. AJAX POST to send data to server
5. JSON usage (stringify for sending, parse for receiving)

```
### LocalStorage Implementation
```javascript
// Save data to localStorage
localStorage.setItem('key', 'value');

// Retrieve data
const value = localStorage.getItem('key');
```

🔥 High Probability Exam Questions

1. Explain `var`, `let`, `const` with scope differences
2. What is a Promise? Explain with example
3. What is Async/Await? How is it different from Promises?
4. What is AJAX? How does it work?
5. What is JSON? Difference between `JSON.stringify` and `JSON.parse`
6. What is an arrow function? How is `this` different?
7. Explain callback function with example
8. What is jQuery? Why use it?
9. What is `$(document).ready()`?
10. What is DOM manipulation? Give example

🧠 Quick Revision Table

Topic	Key Point	One-Liner
`var` vs `let`	Scope difference	"`let` is block-scoped, `var` is function-scoped"
JSON	Data format	"Key-value pairs in double quotes"
Promise	Async object	"Pending → Fulfilled or Rejected"
Async/Await	Syntactic sugar	"Makes async code look sync"
AJAX	Background HTTP	"Update page without full reload"
fetch()	Modern AJAX	"Returns a Promise, use await"
jQuery `$`	DOM library	"Write less, do more"
Callback	Function param	"Call this when done"
localStorage	Browser storage	"Key-value, survives page refresh"
Arrow Function	Short function	"No own this, inherits from parent"
```